Corners

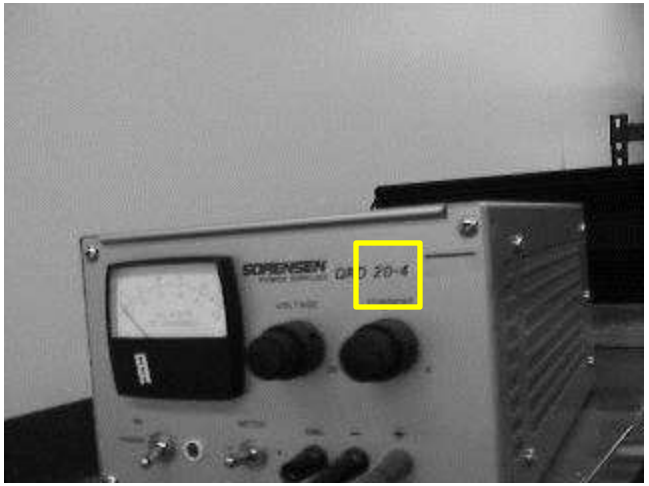
Sum of squared differences

- Say we want to track a patch from image I_0 to image I_1
  - We'll assume that intensities don't change, and only translational motion
  - So we expect that

$$I_1(\mathbf{x}_i + \mathbf{u}) = I_0(\mathbf{x}_i)$$

  - In practice we will have noise, so they won't be exactly equal
- But we can search for the displacement $\mathbf{u}=(x,y)$ that minimizes the sum of squared differences

$$E(\mathbf{u}) = \sum_i w(\mathbf{x}_i)\left[I_1(\mathbf{x}_i + \mathbf{u}) - I_0(\mathbf{x}_i)\right]^2$$



Stability of SSD

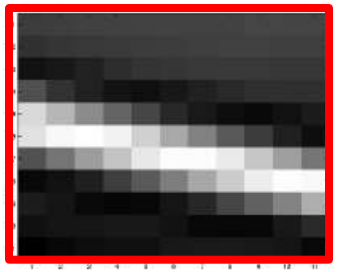
- We would like the SSD score to have a distinct minimum at the correct value of **u**
  - If not, there will be uncertainty in the value of **u**
- The image texture in the patch will determine the stability of the SSD score
  - Bland, featureless patches will not be able to be matched uniquely
- We want to choose patches to track that will be stable
- We can predict how stable a patch will be by comparing its SSD score with itself, at various displacements Δ**u**

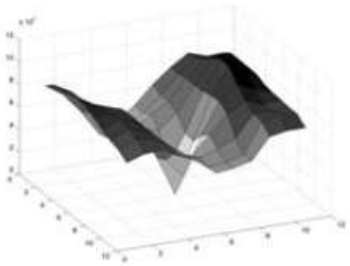
$$E(\Delta\mathbf{u}) = \sum_i w(\mathbf{x}_i)\left[I_0(\mathbf{x}_i + \Delta\mathbf{u}) - I_0(\mathbf{x}_i)\right]^2, \text{ where } \Delta\mathbf{u} = \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix}$$

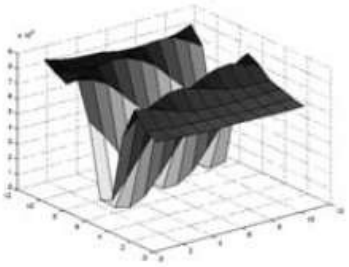
w(**x**) is optional weighting function

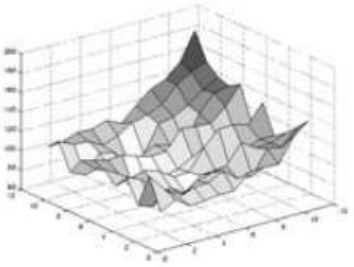
- If the surface *E*(Δ**u**) has a distinct minimum, it is a good feature to track

(a)









(b) (c) (d)

Figure 4.5 Three auto-correlation surfaces $E_{AC}(\Delta u)$ shown as both grayscale images and surface plots: (a) The original image is marked with three red crosses to denote where the auto-correlation surfaces were computed; (b) this patch is from the flower bed (good unique minimum); (c) this patch is from the roof edge (one-dimensional aperture problem); and (d) this patch is from the cloud (no good peak). Each grid point in figures b–d is one value of Δu.

From *Computer Vision: Algorithms and Applications*, Richard Szeliski, 2010, Springer

Stability of SSD Score

- We can estimate the stability of the SSD score
- First, take the Taylor series expansion of the image
  - Recall Taylor series expansion of a function *f(x)* near x_0

$$f(x) = f(x_0) + \left[\frac{df}{dx}\right]_{x_0} (x - x_0) + \cdots$$

  - For vectors **x**, **u**

$$I_0(\mathbf{x}_i + \Delta\mathbf{u}) \approx I_0(\mathbf{x}_i) + \nabla I_0(\mathbf{x}_i) \cdot \Delta\mathbf{u}$$

- Then

$$\begin{aligned} E(\Delta\mathbf{u}) &= \sum_i w(\mathbf{x}_i)\left[I_0(\mathbf{x}_i + \Delta\mathbf{u}) - I_0(\mathbf{x}_i)\right]^2 \\ &\approx \sum_i w(\mathbf{x}_i)\left[I_0(\mathbf{x}_i) + \nabla I_0(\mathbf{x}_i) \cdot \Delta\mathbf{u} - I_0(\mathbf{x}_i)\right]^2 \\ &= \sum_i w(\mathbf{x}_i)\left[\nabla I_0(\mathbf{x}_i) \cdot \Delta\mathbf{u}\right]^2 \end{aligned}$$

where

- $\mathbf{x}_i = (x_i, y_i)^T$ is some image point
- $\Delta\mathbf{u} = (\Delta x, \Delta y)^T$ is the displacement from that point
- $\nabla I_0(\mathbf{x}_i)$ is the gradient of the image at $\mathbf{x}_i$

Stability of SSD Score (continued)

- Expand the SSD score

$$E(\Delta\mathbf{u}) = \sum_i w(\mathbf{x}_i)\left[\nabla I_0(\mathbf{x}_i)\cdot\Delta\mathbf{u}\right]^2$$

$$= \sum_i w(\mathbf{x}_i)\left[\Delta x\frac{\partial I_0(\mathbf{x}_i)}{\partial x} + \Delta y\frac{\partial I_0(\mathbf{x}_i)}{\partial y}\right]^2$$

$$= \sum_i w(\mathbf{x}_i)\left[\Delta x\left(\frac{\partial I_0(\mathbf{x}_i)}{\partial x}\right)^2\Delta x + 2\Delta x\left(\frac{\partial I_0(\mathbf{x}_i)}{\partial x}\right)\left(\frac{\partial I_0(\mathbf{x}_i)}{\partial y}\right)\Delta y + \Delta y\left(\frac{\partial I_0(\mathbf{x}_i)}{\partial y}\right)^2\Delta y\right]$$

$$= \Delta x\left[\sum_i w(\mathbf{x}_i)\left(\frac{\partial I_0(\mathbf{x}_i)}{\partial x}\right)^2\right]\Delta x + 2\Delta x\left[\sum_i w(\mathbf{x}_i)\left(\frac{\partial I_0(\mathbf{x}_i)}{\partial x}\right)\left(\frac{\partial I_0(\mathbf{x}_i)}{\partial y}\right)\right]\Delta y + \Delta y\left[\sum_i w(\mathbf{x}_i)\left(\frac{\partial I_0(\mathbf{x}_i)}{\partial y}\right)^2\right]\Delta y$$

$$= \Delta x\left(w * I_{xx}\right)\Delta x + 2\Delta x\left(w * I_{xy}\right)\Delta y + \Delta y\left(w * I_{yy}\right)\Delta y$$

$$= \begin{pmatrix}\Delta x & \Delta y\end{pmatrix}\begin{pmatrix} w * I_{xx} & w * I_{xy} \\ w * I_{xy} & w * I_{yy}\end{pmatrix}\begin{pmatrix}\Delta x \\ \Delta y\end{pmatrix}$$

- where

$$I_x^2 = \left(\frac{\partial I}{\partial x}\right)^2,\ I_{xy} = \left(\frac{\partial I}{\partial x}\right)\left(\frac{\partial I}{\partial y}\right),\quad I_y^2 = \left(\frac{\partial I}{\partial y}\right)^2$$

Stability of SSD Score (continued)

- The SSD score is

$$E(\Delta\mathbf{u}) = \begin{pmatrix}\Delta x & \Delta y\end{pmatrix}\begin{pmatrix} w * I_{xx} & w * I_{xy} \\ w * I_{xy} & w * I_{yy}\end{pmatrix}\begin{pmatrix}\Delta x \\ \Delta y\end{pmatrix}$$

$$= \Delta\mathbf{u}^T \mathbf{A} \Delta\mathbf{u}$$

$$I_x^2 = \left(\frac{\partial I}{\partial x}\right)^2, I_{xy} = \left(\frac{\partial I}{\partial x}\right)\left(\frac{\partial I}{\partial y}\right),$$

$$I_y^2 = \left(\frac{\partial I}{\partial y}\right)^2$$

- where

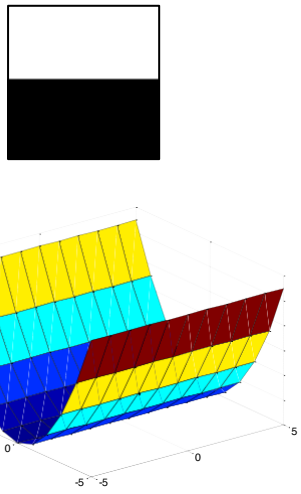
$$\mathbf{A} = w * \begin{pmatrix} I_x^2 & I_{xy} \\ I_{xy} & I_y^2\end{pmatrix}$$

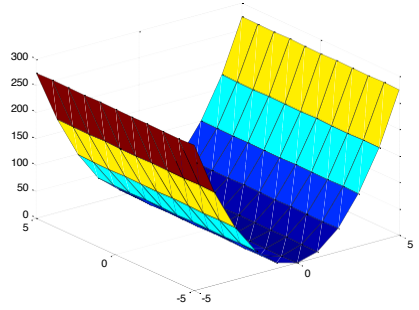
Note – the "" indicates correlation of w with each element of* **A**

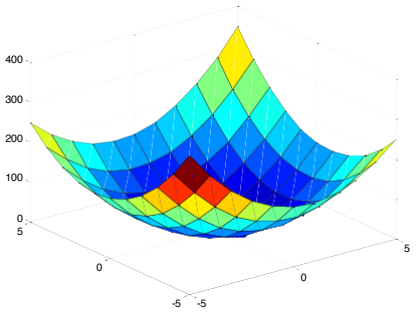
- *w* is a small *NxN* averaging filter such as a box filter
- it essentially averages the values over a small local neighborhood

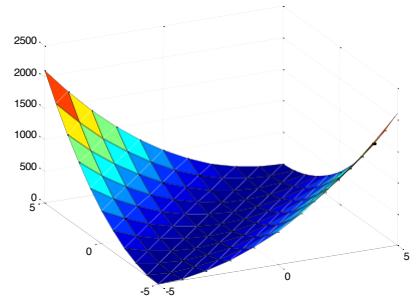
- **A** can tell us how stable the SSD score is, at a given point

Example Patches









0	0
0	22

22	0
0	0

12	1
1	12

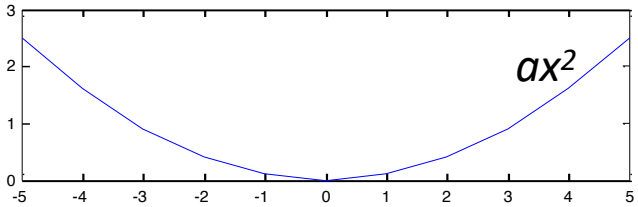
21	-21
-21	21

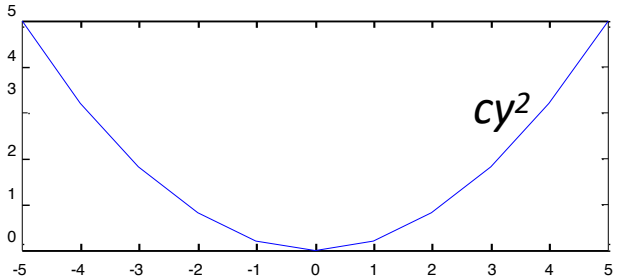
$$\mathbf{A} = w * \begin{pmatrix} I_x^2 & I_{xy} \\ I_{xy} & I_y^2 \end{pmatrix}$$

Stability of SSD Score (continued)

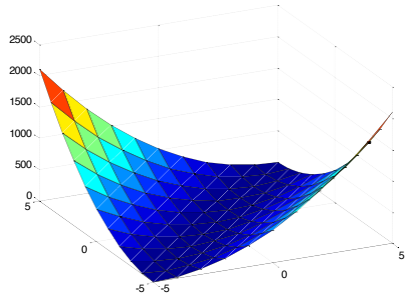
- $E=\Delta\mathbf{u}^T\,\mathbf{A}\,\Delta\mathbf{u}$ is a second order polynomial
  - Has the form $E(x,y)=ax^2+bxy+cy^2$
  - The matrix **A** gives the curvature of the surface

$$\mathbf{A}=\begin{pmatrix} a & b \\ b & c \end{pmatrix}$$





- A large value of *a* means that *E* curves up sharply as *x* varies
  - Similarly, large *c* means that *E* curves up sharply as *y* varies
  - Both *a,c* large mean that we have a good minimum in *E*
- Problem – if *b* is not zero, the surface may be flat along the diagonal



Estimating stability (continued)

- Recall eigenvalues

  $\mathbf{A}\,\mathbf{v}_i = \lambda_i \mathbf{v}_i$

  where λ_i is the ith eigenvalue, $\mathbf{v}_i$ is the ith eigenvector
  - We can write

$$\mathbf{A}\begin{pmatrix}\mathbf{v}_1 & \mathbf{v}_2\end{pmatrix} = \begin{pmatrix}\lambda_1 & 0 \\ 0 & \lambda_2\end{pmatrix}\begin{pmatrix}\mathbf{v}_1 & \mathbf{v}_2\end{pmatrix}$$

  - Or $\mathbf{AR}=\mathbf{\Lambda R}$

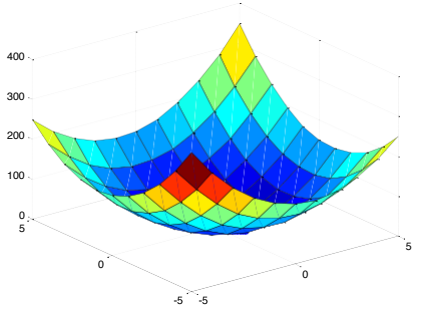
- Now, $\mathbf{R} = (\mathbf{v}_1, \mathbf{v}_2)$ is an orthonormal matrix (its columns are mutually orthogonal, and all unit vectors). So $\mathbf{R}^T = \mathbf{R}^{-1}$

- So $\mathbf{R}^T\mathbf{AR}=\Lambda$
  - Where the columns of **R** are the eigenvectors of **A**
  - $\mathbf{\Lambda}$ is a diagonal matrix whose elements are the eigenvalues of **A**

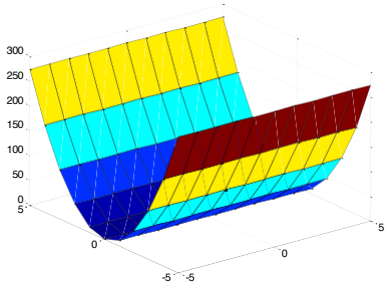
Estimating stability (continued)

- **R** is a rotation matrix
  - If we rotate the coordinate system to $\Delta\mathbf{u}=\mathbf{R}\,\Delta\mathbf{v}$
  - Then $E = \Delta\mathbf{u}^T\,\mathbf{A}\,\Delta\mathbf{u} = (\Delta\mathbf{v}^T\mathbf{R}^T)\,\mathbf{A}\,(\mathbf{R}\,\Delta\mathbf{v}) = \Delta\mathbf{v}^T\,\Lambda\,\Delta\mathbf{v}$
  - Which has the form $E(v_1,v_2) = \lambda_1 v_1^2 + \lambda_2 v_2^2$

- So if both eigenvalues are large, we have a good minimum (there is no diagonal component)

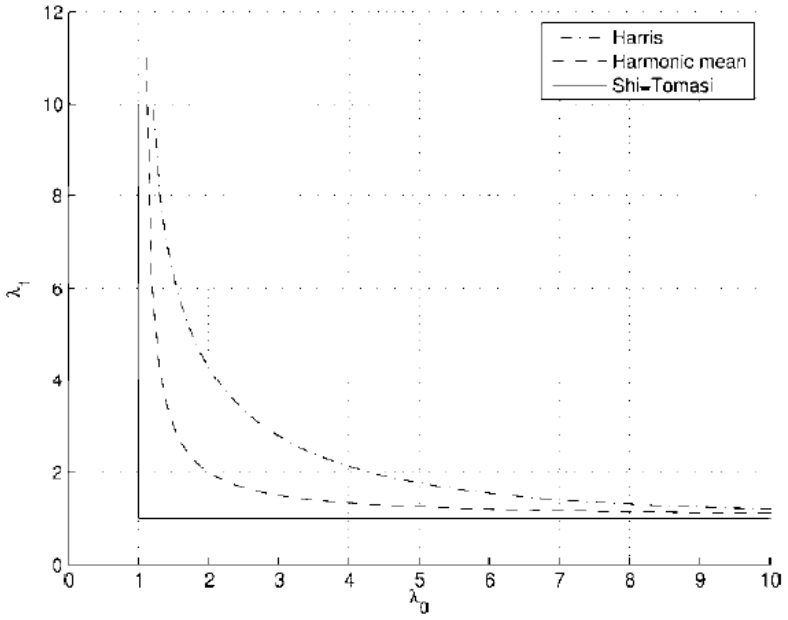


- If one eigenvalue is low, surface is flat along that direction



Possible interest point measures

- Look at the smallest eigenvalue – if it is sufficiently large, then we have a candidate point (**Shi-Tomasi**)



From *Computer Vision: Algorithms and Applications*, Richard Szeliski, 2010, Springer

- Alternative measures that don't require square roots:

(**Harris**)

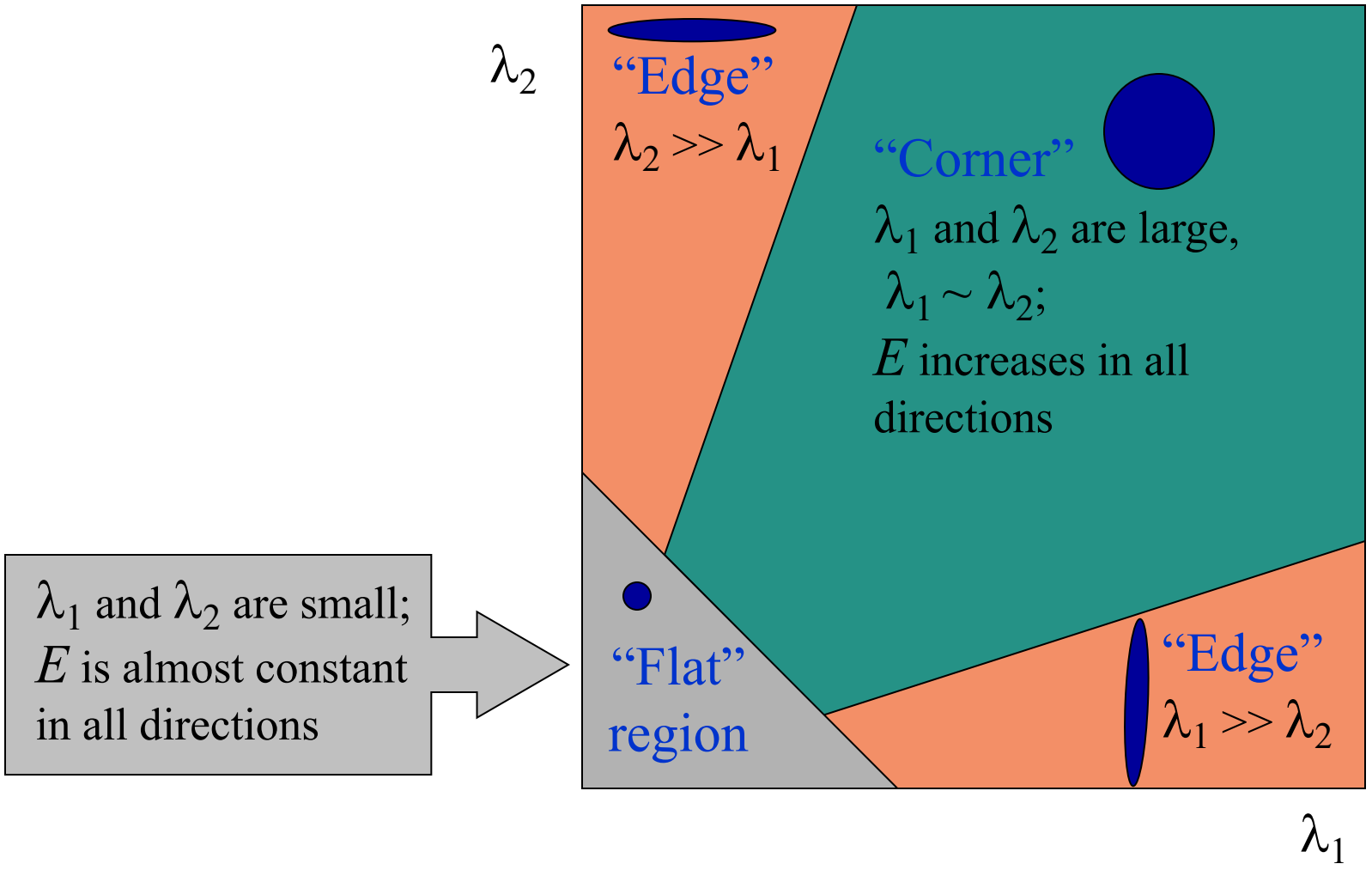
$$\det(\mathbf{A}) - \alpha\ \mathrm{tr}(\mathbf{A})^2 = \lambda_0\lambda_1 - \alpha(\lambda_0 + \lambda_1)^2$$

(**Harmonic mean**)

$$\frac{\det(\mathbf{A})}{\mathrm{tr}(\mathbf{A})} = \frac{\lambda_0\lambda_1}{\lambda_0 + \lambda_1}$$

Interpreting the eigenvalues

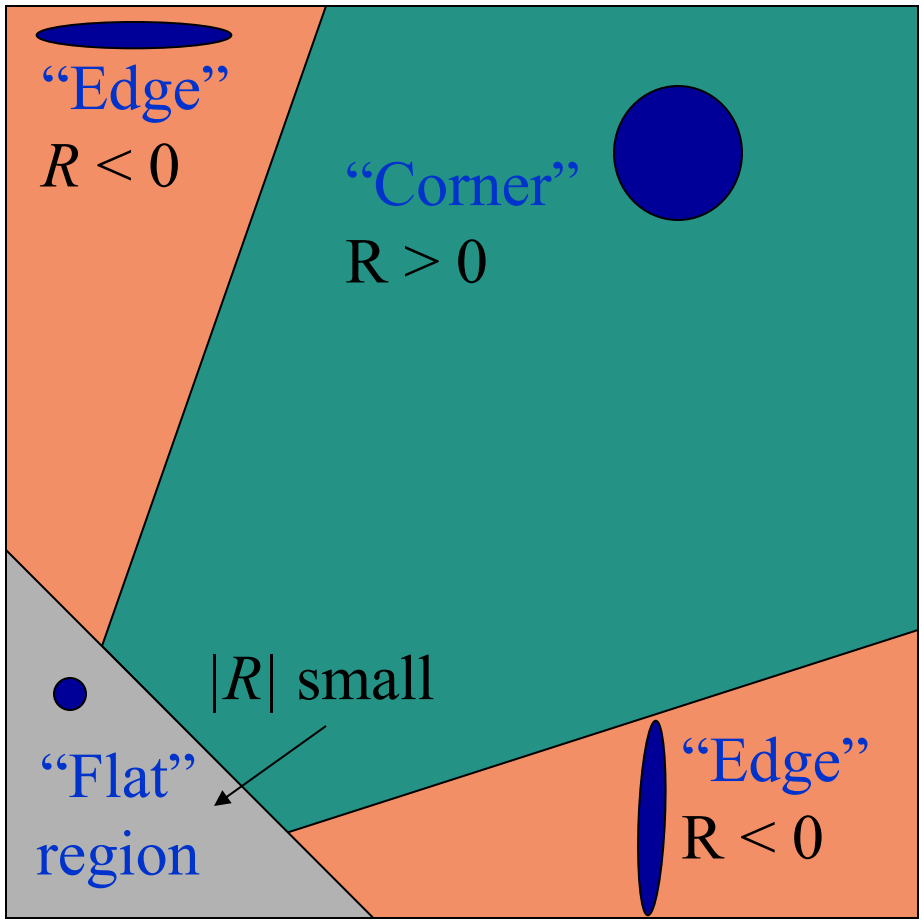
Classification of image points using eigenvalues of *M*:



Corner response function

$$R = \det(M) - \alpha \,\mathrm{trace}(M)^2 = \lambda_1\lambda_2 - \alpha(\lambda_1 + \lambda_2)^2$$

α: constant (0.04 to 0.06)



Python Corner Interest Operator

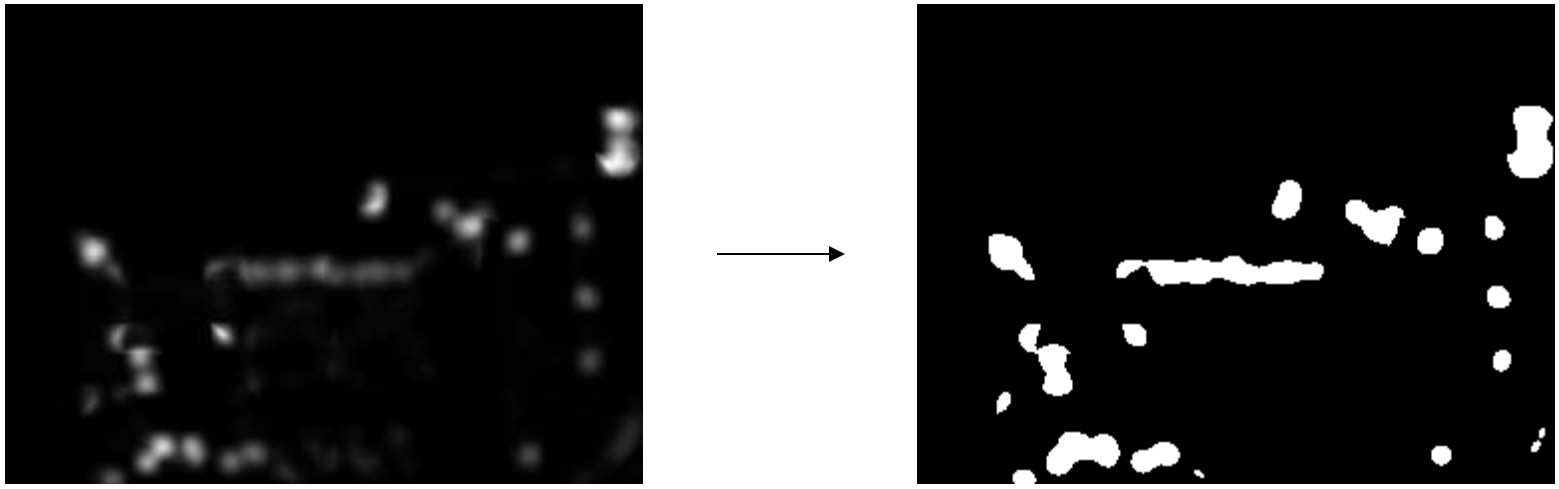
```
img = cv2.imread('test000.jpg')

img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Display image
plt.imshow(img_gray, cmap='gray')
plt.show()

# Apply Gaussian blur
img_gray = np.float32(img_gray)

sigma = 1
img_blur = cv2.GaussianBlur(img_gray, (13,13), sigma)

plt.imshow(img_blur, cmap='gray')
plt.show()

# Compute the gradient components
Gx = cv2.Sobel(img_blur,cv2.CV_64F,1,0,ksize=5)
Gy = cv2.Sobel(img_blur,cv2.CV_64F,0,1,ksize=5)

# Compute the outer product g'g at each pixel
Gxx = Gx * Gx
Gxy = Gx * Gy
Gyy = Gy * Gy
```

Python Corner Interest Operator (continued)

```
# Size of neighborhood over which to compute corner features.
N = 13
w = np.ones(N) # The neighborhood
# Sum the G's over the window size.
A11 = cv2.filter2D(Gxx, cv2.CV_64F, w)
A12 = cv2.filter2D(Gxy, cv2.CV_64F, w)
A22 = cv2.filter2D(Gyy, cv2.CV_64F, w)

# At each pixel (x,y), we have the 2x2 matrix
# [A11(x,y) A12(x,y)
# A21(x,y) A22(x,y)]
# Of course, A21 = A12.

# Compute the interest score: det(A) - alpha * trace(A)^2
alpha = 0.06
detA = A11 * A22 - A12**2 # Computes det(A) at each pixel
traceA = A11 + A22 # Computer trace(A) at each pixel
s = detA - alpha * traceA**2 # The interest score


plt.imshow(s)
plt.colorbar()
plt.show()
```

Finding Local Maxima

- Simply finding all points greater than a threshold will yield too many points



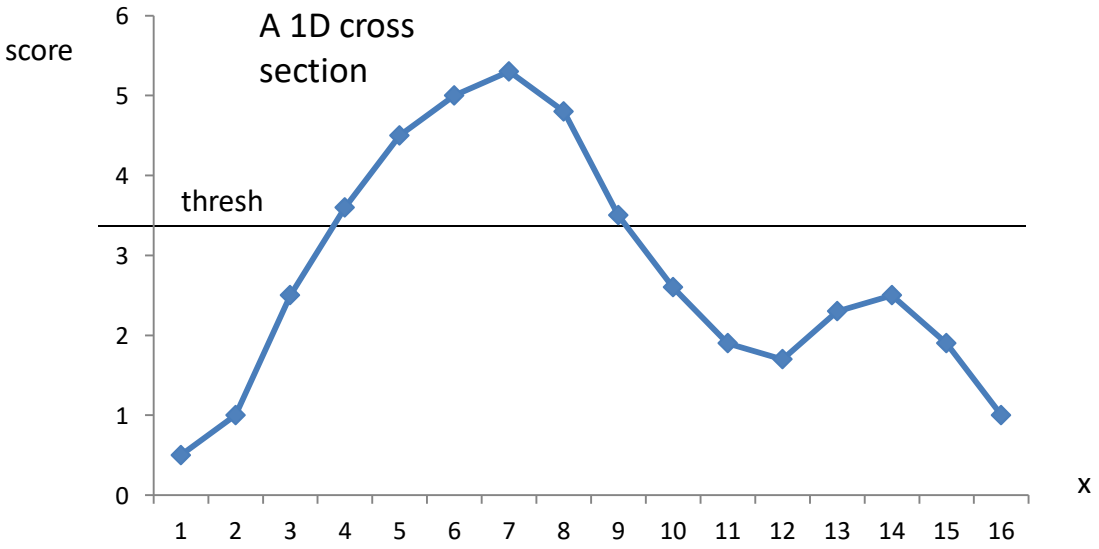
interest scores

interest scores, thresholded

- We should find points that are
  - greater than the threshold
  - a local maximum

Finding Local Maxima

- If peaks are smooth, you can test each point to see if it is higher than its neighbors



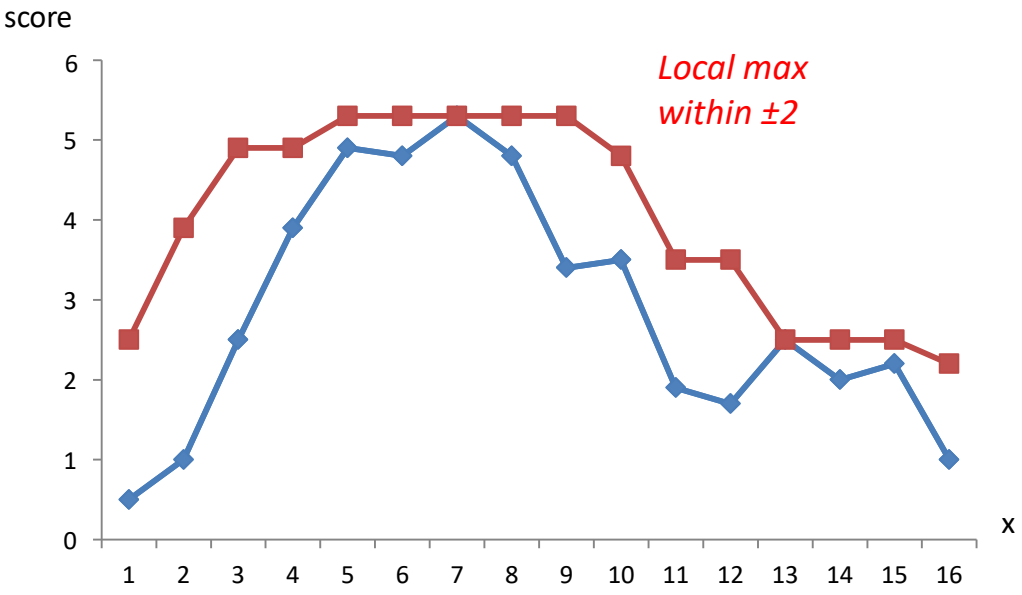
In 2D, check 4 nearest neighbors

- Matlab code

```
Lmax = false(size(S));
Lmax(S(2:end-1, 2:end-1) = ...
    ( S(2:end-1, 2:end-1) > S(3:end, 2:end-1) ...
    & S(2:end-1, 2:end-1) > S(1:end-2, 2:end-1) ...
    & S(2:end-1, 2:end-1) > S(2:end-1, 3:end) ...
    & S(2:end-1, 2:end-1) > S(2:end-1, 1:end-2) );
```

Finding Local Maxima (continued)

- Problem – what if you don't have smooth peaks?
  - Too many detected features



- Instead, find points that are local *regional* maxima – they are the largest points within a region (e.g., square of size W)
- Can do by calculating the largest value within distance N (this is a *dilation*)
- Then find those points that equal the local maxima

```
Smax = (S==imdilate(S, ones(N)));
```

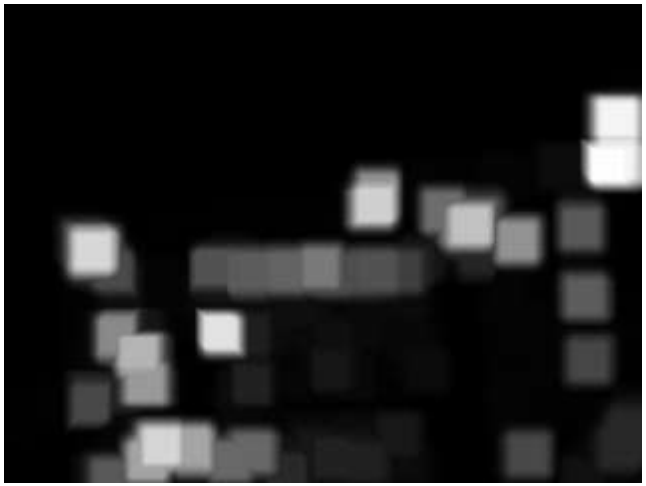
Finding Local Maxima (continued)

Score array

S

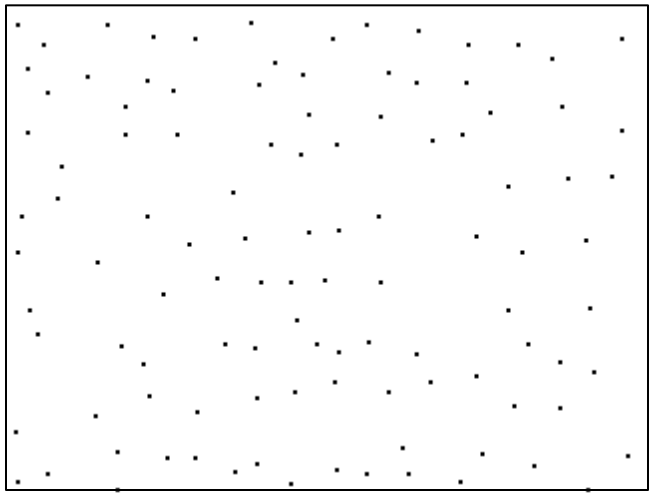
Score array dilated with square of side 15

Sd

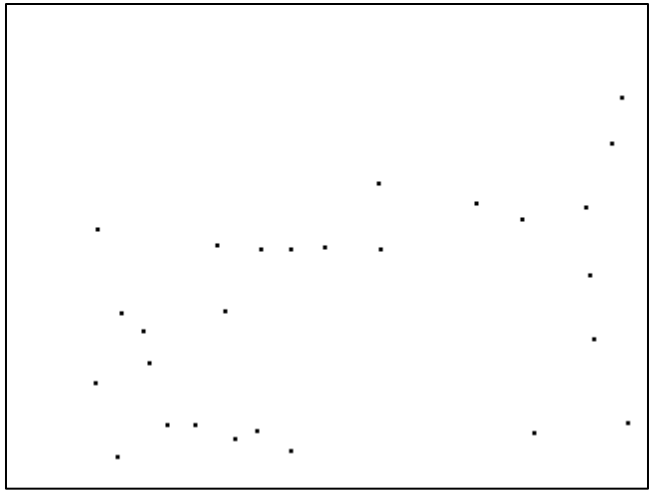


Points where S=Sd

Smax



Points where S=Sd and are > threshold



Python/OpenCV

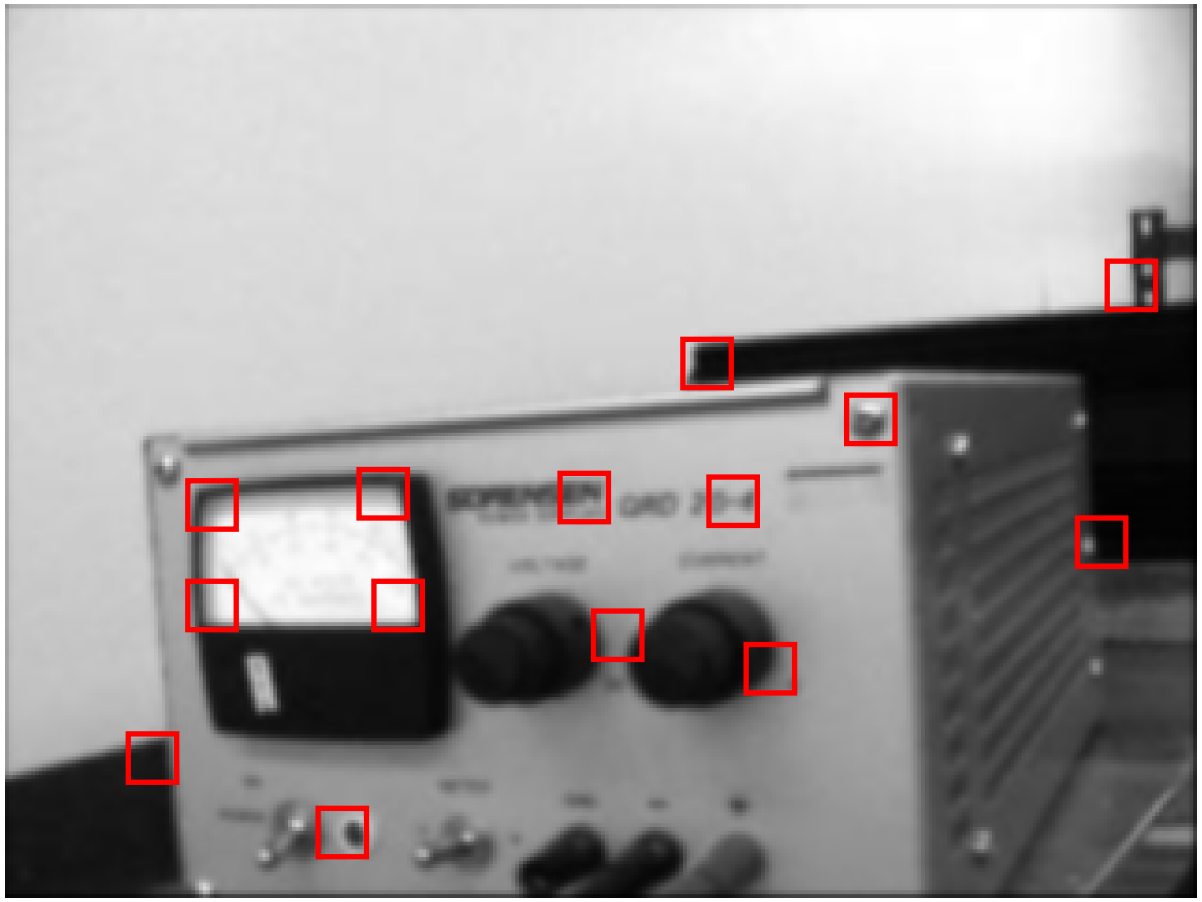
```
% Choose the suppression radius, for non-maxima suppression
r = N;

% Find local maxima within each neighborhood of radius 2r
Lmax = (s==imdilate(s, strel('disk',2*r)));

% Note - we don't want to detect points too close to the border, so just
% zero out everything near the border.
Lmax(1:N,:) = false;
Lmax(:,1:N) = false;
Lmax(end-N:end,:) = false;
Lmax(:,end-N:end) = false;

% Get a list of the indices of all the potential interest points
[rows cols] = find(Lmax);

% Get the values of those interest points
vals = s(Lmax);
```

Look at range of scores

Identifying interest points

- Once you have the scores, you can decide which points to identify as interest points
  - Could choose the points with the top *M* scores
  - Or, choose all points with scores greater than a threshold

```
% Identify interest points above a threshold, and draw
% a box around them, of size NxN
for i=1:length(vals)
    if vals(i) > threshold
        x = cols(i);
        y = rows(i);

        rectangle('Position', [x-N/2 y-N/2 N N], ...
            'EdgeColor', 'r', ...
            'Linewidth', 2.0); % default is 0.5

        text(x,y, sprintf('%d', i), ...
            'Color', 'r', ...   % label with id number
            'FontSize', 16);    % default is 10
    end
end
```



Corner interest points:

- Window size = 13
- Threshold = 100
- Local maxima within 13 pixels

Matching interest points

- Once an interest point is found, we can find its match in another image by minimizing the SSD
- Under certain conditions this is equivalent to maximizing the cross-correlation score

$$E(\mathbf{u}) = \sum_i \left[I_1(\mathbf{x}_i + \mathbf{u}) - I_0(\mathbf{x}_i) \right]^2$$

$$= \sum_i I_1(\mathbf{x}_i + \mathbf{u})^2 - 2\sum_i I_1(\mathbf{x}_i + \mathbf{u}) I_0(\mathbf{x}_i) + \sum_i I_0(\mathbf{x}_i)^2$$

- The middle term is the cross-correlation of I_1 and I_0 … it is high when I_1 matches I_0
- Cross-correlation can be done very efficiently (in the frequency domain)

Vector representation of correlation

- Correlation is a sum of products of corresponding terms

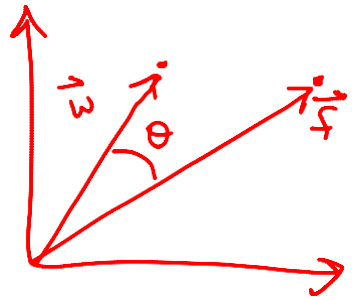
$$c(x,y) = \sum_{s=-m/2}^{m/2} \sum_{t=-n/2}^{n/2} w(s,t)\, f(x+s, y+t)$$
$$= w(x,y) \otimes f(x,y)$$

- We can think of correlation as a dot product of vectors **w** and **f**

$$c = w_1 f_1 + w_2 f_2 + \ldots + w_{mn} f_{mn} = \mathbf{w} \cdot \mathbf{f}$$

- If images w and f are similar, their vectors (in *mn*-dimensional space) are aligned

$$c = \left|\mathbf{w}\right|\left|\mathbf{f}\right| \cos\theta$$



Correlation can do matching

- Let *w(x,y)* be a template sub-image
- Try to find an instance of *w* in image *f(x,y)*

- The correlation score *c(x,y)* is high where *w* matches *f*

$$c(x,y) = \sum_{s=-m/2}^{m/2} \sum_{t=-n/2}^{n/2} w(s,t)\, f(x+s, y+t)$$

$$= w(x,y) \otimes f(x,y)$$

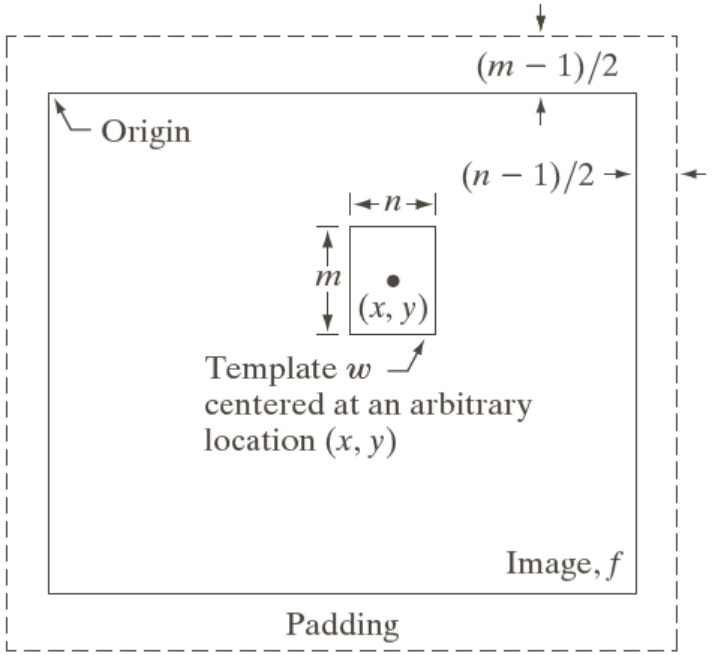


FIGURE 12.8
The mechanics of template matching.

Template Matching (continued)

- Since f is not constant everywhere, we need to normalize

$$c = \frac{\mathbf{w} \cdot \mathbf{f}}{\left|\mathbf{w}\right|\left|\mathbf{f}\right|} = \cos\theta$$

- We can get better precision by subtracting off means

$$c(x,y) = \frac{\sum_{s,t}[w(s,t)-\bar{w}][f(x+s,y+t)-\bar{f}]}{\left\{\sum_{s,t}[w(s,t)-\bar{w}]^2 \sum_{s,t}[f(x+s,y+t)-\bar{f}]^2\right\}^{1/2}}$$

This is the normalized cross correlation coefficient

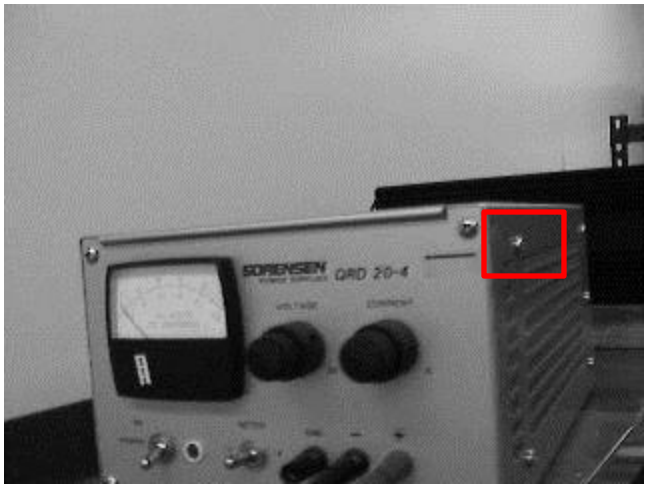
Perfectly opposite

Perfect match

Range: -1.0 … +1.0

Example

- Track a point from one image to the next
  - `imcrop` – get template
  - `normxcorr2` – normalized cross correlation



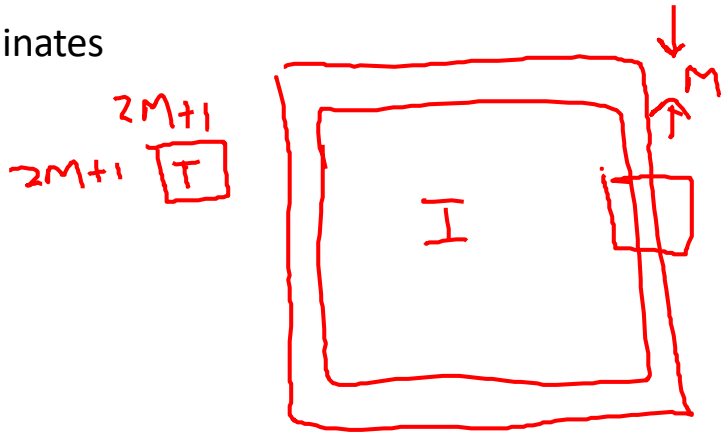
Note on normxcorr2

- The Matlab function normxcorr2 correlates a template T with image I
  - To find the maximum, can do

```
C = normxcorr2(T,I); % Do normalized cross correlation
cmax = max(C(:));
[y2 x2] = find(C==cmax);
```

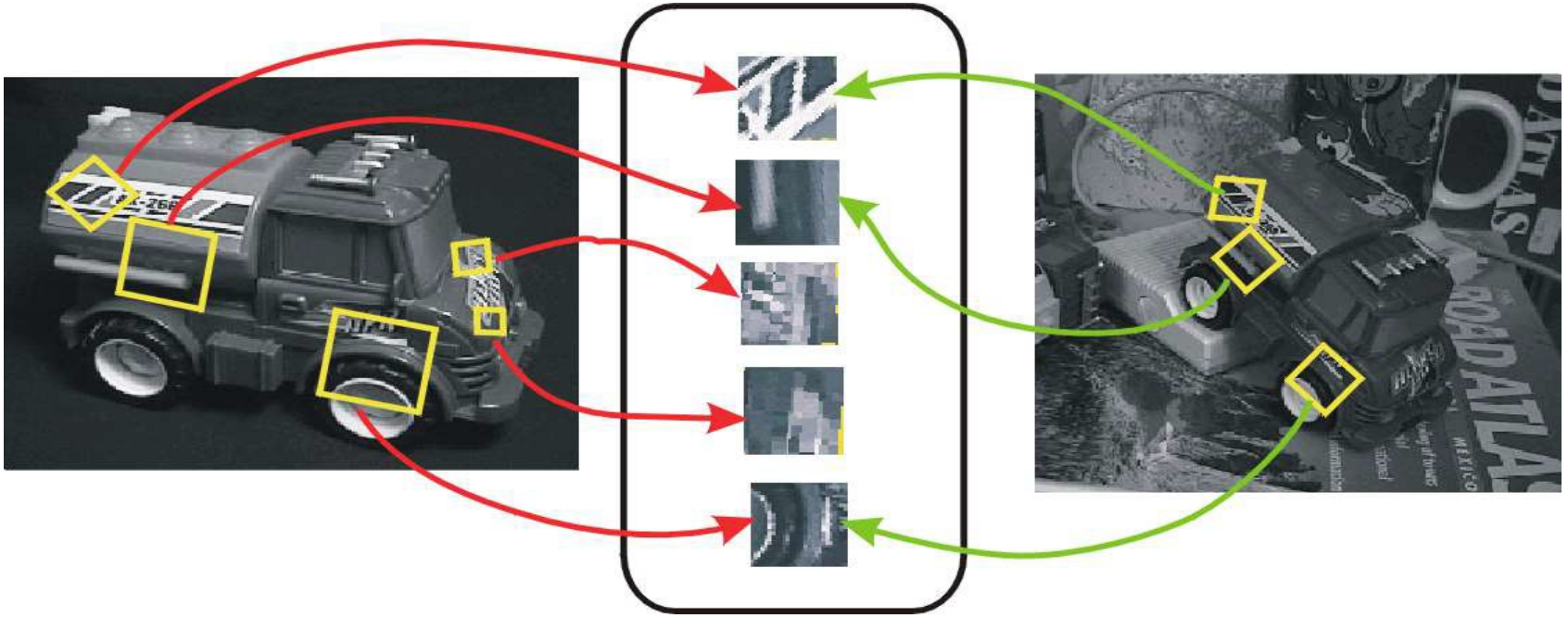
- Assume T is size (2*M+1)x(2*M+1)
  - Then the resulting score image is bigger than I, by M rows and M columns along each side and top and bottom
  - You should subtract off M from the coordinates

```
y2 = y2-M;
x2 = x2-M;
```



Handling More General Viewpoint Changes

- Cross correlation assumes translation only
  - But appearance of image patches changes with rotation, scale, viewpoint
- Need features that are invariant to translation, rotation, scale, and viewpoint



Affine Corner Detector

- Affine transformation $\mathbf{I}_2\left(\mathbf{Ax}+\mathbf{d}\right)=\mathbf{I}_1(\mathbf{x})$
- where $\mathbf{A}=\begin{pmatrix} a_{xx} & a_{xy} \\ a_{yx} & a_{yy} \end{pmatrix}$
  - **d** is the translation, **A** is the deformation matrix
- This can account for rotation and scale changes
- Also viewpoint changes, if planar patch is small compared to distance from camera

KLT Tracker

- Find interest points in the first frame
- Track (assuming translation only) points from each frame to subsequent frame
- Test consistency with original frame by computing affine transformation

Figure 1: Three frame details from Woody Allen's *Manhattan*. The details are from the 1st, 11th, and 21st frames of a subsequence from the movie.

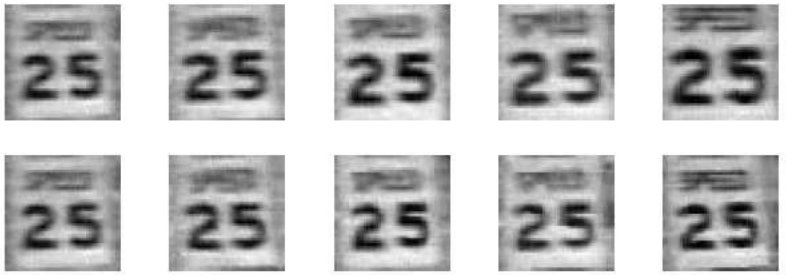


Figure 2: The traffic sign windows from frames 1,6,11,16,21 as tracked (top), and warped by the computed deformation matrices (bottom).

from: Shi and Tomasi, "Good Features to Track", *IEEE CVPR* 1994